

Phase 2 - Formal PDF

Ap Calderon, Calista Ruiz, Erick Rayas, Linh Tran, Raul Munoz

CPSC 351

California State University, Fullerton

Professor Sameer Nagar

29 April 2026

Storyline

We are in an age in which video games have become commonplace for people of all ages. There are many game systems, each with unique games, allowing users to play whatever is fun for them. However, one thing all video games share is that, when playing an Online game, the user must connect to the game server to play with other online players. For our project, imagine we have just finished creating an online open-world game and must decide how users will join the servers and start playing. This is important because we are planning for this game to be a success, which means the server's capacity will be an issue. While there are currently no plans for expansion, we want to ensure that each user understands how the game queue will work when entering the game servers.

Players are logging into the server to play. Scheduling in this setting is important since due to the finite amount of players a server can host at a time, there's bound to be more players than the server's able to handle. Therefore, there must be a fair and efficient way to choose which players to let in and which to transfer to another server. For this reason, we have decided that these three options would make the most sense. First, the First Come First Serve algorithm was chosen because this is how many game servers operate today, meaning whatever user joins first gets access first. Second, Shortest Job First was considered, which admits players into the server based on how long it will take the server to fully process user information. It will admit players with the quickest execution time and ignore users who carry a large execution time. Finally, the Priority Scheduling algorithm. This means we are planning on creating a premium service that allows users to cut the line, who enter before non-premium users, getting into the server first. These algorithms highlight the different methods in which we can ensure either happiness and/or profit from users when connecting to the game server.

Algorithms (Non-Preemptive)

- **FCFS**: Players are allowed server access strictly in the order they join the queue. This is simple to implement and easy to understand, but if an early-arriving player has a long load time, every player behind them is blocked and forced to wait. This would cause load times to vary largely between players, creating bottlenecks.
- **SJF**: The server admits players with the fastest connection times first, clearing the queue as fast as possible and maximizing throughput. The downfall is that players with slow internet or large downloads get pushed back indefinitely, causing starvation. Fairness is thrown out with this method, but it will allow the majority of people to quickly gain access to servers.
- **Priority Scheduling**: Priority level is assigned to players based on their account tier. Premium members are placed at the front of the admission queue, leaving free players or new players in the back. In the event of multiple users having the same priority, scheduling will fall back to whoever arrives first. The risk of starvation of free players would cause casual gamers to stray away from the game completely.

Initial Input Design

Process (Unique and Individual Player)	Arrival (Start establishing a connection to the server)	Burst (Time it takes for the server to authenticate the user)	Priority (Premium users who are given access to the front of the line to join the server)
User1 [Free]	0	6	2
User2 [Free]	1	3	2
User3 [Priority]	2	5	1
User4 [Free]	4	1	2
User5 [Priority]	6	4	1

```
Server started at 00:00
00:00 User1 [Member] has joined the server queue
00:01 User2 [Member] has joined the server queue
00:02 User3 [VIP] has joined the server queue
00:04 User4 [Member] has joined the server queue
00:06 User5 [VIP] has joined the server queue
```

First Come First Serve:

Scheduling Order:

User1 → User2 → User3 → User4 → User5

- **0-6:** User1 (0), User2 (3), User3 (5), User4 (1), User5 (4)
- **6-9:** User2 (0), User3 (5), User 4(1), User5 (4)
- **9-14:** User3 (0), User4 (1), User5 (4)
- **14-15:** User4 (0), User5 (4)
- **15-19:** User5 (0)

Process (Unique User)	Arrival	Burst	CT	WT: TAT-BT	TAT: CT-AT
User1 [Free]	0	6	6 [6+0]	0	6
User2 [Free]	1	3	9 [6+3]	5	8
User3 [Priority]	2	5	14 [9+5]	7	12
User4 [Free]	4	1	15 [14+1]	10	11
User5 [Priority]	6	4	19 [15+4]	9	13

Average WT: $(0+5+7+10+9 = 31/5 = 6.2)$ | **Average TAT:** $(6+8+12+11+13 = 50/5 = 10)$

```
--- First Come First Serve ---
00:06 User1      [Member] left after waiting 0s (server processed in 6s) | TAT: 6s | loaded at: 00:06
00:09 User2      [Member] left after waiting 5s (server processed in 3s) | TAT: 8s | loaded at: 00:09
00:14 User3      [VIP]    left after waiting 7s (server processed in 5s) | TAT: 12s | loaded at: 00:14
00:15 User4      [Member] left after waiting 10s (server processed in 1s) | TAT: 11s | loaded at: 00:15
00:19 User5      [VIP]    left after waiting 9s (server processed in 4s) | TAT: 13s | loaded at: 00:19

Avg Wait Time: 6.2s | Avg Turnaround Time: 10.0s | Avg Completion Time: 12.6s
```

The terminal output confirms the scheduling order and calculated metrics. User1 is processed immediately upon arrival with 0 wait time. Each player waits for the one before them to finish, resulting User4 experiencing the longest individual wait time at 10 seconds, despite having the shortest burst time of only 1 second.

Shortest Job First:

Scheduling Order:

User1 → User4 → User2 → User5 → User3

- 0-6: User1 (0), User2 (3), User3 (5), User4 (1), User5 (4)
- 6-7: User4 (0), User2 (3), User3 (5), User5 (4)
- 7-10: User2 (0), User3 (5), User5 (4)
- 10-14: User5 (0), User3 (5)
- 14-19: User3 (0)

Process (Unique User)	Arrival	Burst	CT	WT: TAT-BT	TAT: CT-AT
User1 [Free]	0	6	6 [6+0]	0	6
User2 [Free]	1	3	10 [7+3]	6	9
User3 [Priority]	2	5	19 [14+5]	12	17
User4 [Free]	4	1	7 [6+1]	2	3
User5 [Priority]	6	4	14 [10+4]	4	8

Average WT: $[(0+6+12+2+4)/5] = 4.8$ | **Average TAT:** $[(6+9+17+3+8)/5] = 8.6$

```
-- Shortest Job First ---
00:06 User1    [Member] left after waiting 0s (server processed in 6s) | TAT: 6s | loaded at: 00:06
00:10 User2    [Member] left after waiting 6s (server processed in 3s) | TAT: 9s | loaded at: 00:10
00:19 User3    [VIP]    left after waiting 12s (server processed in 5s) | TAT: 17s | loaded at: 00:19
00:07 User4    [Member] left after waiting 2s (server processed in 1s) | TAT: 3s | loaded at: 00:07
00:14 User5    [VIP]    left after waiting 4s (server processed in 4s) | TAT: 8s | loaded at: 00:14
Avg Wait Time: 4.8s | Avg Turnaround Time: 8.6s | Avg Completion Time: 11.2s
```

The terminal output confirms that once User1 finished at $t=6$, the algorithm selects User4 next due to its burst time of 1 second rather than continuing in arrival order. SJF's reordering system is what produces the best system-wide averages out of the three algorithms.

Priority Scheduling:

Scheduling Order: User1 → User3 → User5 → User2 → User4

- 0-6: User1 (0), User2 (B:3, P: 2), User3 (B:5, P:1), User4 (B:1, P:2), User5 (B:4, P:1)
- 6-11: User3 (0), User2 (B:3, P: 2), User4 (B:1, P:2), User5 (B:4, P:1)
- 11-15: User5 (0), User2 (B:3, P: 2), User4 (B:1, P:2)
- 15-18: User2 (0), User4 (B:1, P:2)
- 18-19: User4 (0)

Process (Unique User)	Arrival	Burst	Priority	CT	WT: TAT-BT	TAT: CT-AT
User1 [Free]	0	6	2	6 [6+0]	0	6
User2 [Free]	1	3	2	18 [15+3]	14	17
User3 [Priority]	2	5	1	11 [6+5]	4	9
User4 [Free]	4	1	2	19 [18+1]	14	15
User5 [Priority]	6	4	1	15 [11+4]	5	9

Average WT: $[(0+14+4+14+5)/5] = 7.4$ | **Average TAT:** $[(6+17+9+15+9)/5] = 11.2$

```
--- Priority Scheduling ---
00:06 User1      [Member] left after waiting 0s (server processed in 6s) | TAT: 6s | loaded at: 00:06
00:18 User2      [Member] left after waiting 14s (server processed in 3s) | TAT: 17s | loaded at: 00:18
00:11 User3      [VIP]    left after waiting 4s (server processed in 5s) | TAT: 9s | loaded at: 00:11
00:19 User4      [Member] left after waiting 14s (server processed in 1s) | TAT: 15s | loaded at: 00:19
00:15 User5      [VIP]    left after waiting 5s (server processed in 4s) | TAT: 9s | loaded at: 00:15

Avg Wait Time: 7.4s | Avg Turnaround Time: 11.2s | Avg Completion Time: 13.8s
```

The terminal output confirms that after User1 completes at t=6, the algorithm skips User2 and User4 in favor of User3, because of their VIP status. Both VIP users (User3 and User5) have wait times of 4 and 5 seconds, while the free users both wait 14 seconds, the longest wait time across all three algorithms.

Algorithm Comparisons

<u>System-Level Performance</u>			
Algorithm	FCFS	SJF	Priority
Avg Wait Time	6.2s	4.8s	7.4s
Avg Turnaround	10.0s	8.6s	11.2s
Avg Completion	12.6s	11.2s	13.8s

FCFS vs SJF

Both algorithms treat all users equally in terms of priority, but differ in execution. FCFS processes players based on arrival time, so a player with a long burst time can bottleneck everyone who arrives after. SJF addresses this by ordering the queue to run shorter jobs first, which is why it achieved a lower average wait time (4.8s v 6.2s) and turnaround time (8.6s v 10.0s).

SJF vs Priority

SJF works for the benefit of the whole system, while Priority works for certain users (VIPs). SJF produced the best average wait time across all three algorithms, but it distributed that benefit unfairly. It punishes users with long burst times like User3 (WT of 12s). Priority scheduling had the worst average wait time (7.4s), but treated the users with premiums with a better experience than casual players. The VIP users experienced wait times of only 4-5s, while the casual players experienced wait times of up to 14s.

FCFS vs Priority

FCFS is fair in the order that players arrive, while Priority will intentionally be unfair to certain users. Similar to the previous comparison, Priority does a good job of ensuring that VIP users have a better experience, but are very unfair to regular users. VIP users cut their average wait time almost in half compared to FCFS, while free users' wait times more than doubled.

Overall

SJF wins on every system-level metric, but averages alone can be misleading. A low average wait time does not mean every user had a good experience.

Visualize User Experience

From a player's point of view:

SJF is great for short and high burst time players but not so great if you have a longer burst time.

Priority is great if you're a VIP but terrible if you're a regular user.

FCFS doesn't discriminate against any players but you still might have to wait.

Fairness vs Performance

FCFS → Fair but Inefficient

SJF → Efficient but unfair to players with high burst times

Priority → Fair to the people we want to be fair too.

Graphical Real-Life Example

If you had a real multiplayer game server you would ask yourself these questions:

Do you want your game to be fair? FCFS

Do you want your game to be fast? SJF

Do you want to monetize your game? Priority Scheduling

Code Results:

As you can see from the results, each algorithm affects both individual players as well as the performance of the system as a whole.

Even though all 3 algorithms are servicing the exact same players, the order in which the players are serviced drastically affects the spread of waiting time between users.

FCFS saw some average numbers but began to back up waiting players as new ones arrived because it only services players in order.